

MVNIDS: A multiview-based network intrusion detection system

Sunit Kumar Nandi [✉], Ritesh Ratti [✉], Sanasam Ranbir Singh [✉], Sukumar Nandi [✉]

Indian Institute of Technology Guwahati Guwahati, Assam, India

ARTICLE INFO

Keywords:

Intrusion detection systems
Network attacks
Network security
Unsupervised machine learning
Self supervised learning
Autoencoder network

ABSTRACT

Traditional Network Intrusion Detection Systems (NIDS) predominantly rely on signature-based and supervised learning approaches that require large volumes of labeled attack data. However, generating such labeled data is costly, time-consuming, and often impractical, especially in the presence of class imbalance and zero-day attacks. To address these limitations, this paper proposes MVNIDS, a Multiview-based Network Intrusion Detection System employing a self-supervised learning paradigm. The proposed method constructs three complementary views from raw packet capture data, namely, Network View, Flow View, and Image View, each capturing distinct protocol, temporal, and structural characteristics of network traffic. Independent autoencoder models are trained on benign samples for each view, and their reconstruction errors are fused through a majority-voting mechanism to automatically generate “Attack” and “Benign” pseudo-labels. These labels are subsequently used to train a binary classifier for final intrusion detection. Experimental evaluation on the CICIDS2018 dataset, focusing on FTP BruteForce and UDP DoS attacks, demonstrates that MVNIDS outperforms most view-specific and supervised baselines, achieving up to 98.3 % F1-score and 98.5 % accuracy. The multiview representation enhances detection robustness and enables effective identification of zero-day and variant attacks, highlighting MVNIDS as a scalable, computationally efficient, and generalizable framework for modern network security applications.

1. Introduction

Network Intrusion Detection Systems (NIDS) play a critical role in safeguarding modern communication infrastructures against a growing range of cyber threats. These threats are easily generated by attackers using open-source utilities like LOIC [1], Slowloris [2], Nmap [3], etc. and can hamper overall communication between entities and affect system performance. However, most existing NIDS frameworks remain heavily dependent on signature-based and supervised learning techniques for attack detection. Signature-based systems rely on manually engineered rules or known attack patterns, while supervised learning models depend on large volumes of labeled data for training. In both cases, the availability and quality of labeled data directly determine system performance.

The reliance on labeled datasets presents several challenges. First, curating labeled attack data is costly and time-consuming, as it requires extensive domain expertise and continuous updates to accommodate new or evolving attack variants. Second, even in widely used benchmark datasets, the number of attack samples is significantly smaller than that of benign samples, resulting in high class imbalance. This imbalance leads to biased models that tend to favor normal traffic, often missing subtle or rare malicious patterns. Moreover, in real-world network environments, new or previously unseen attacks, commonly referred to as

zero-day attacks are frequent and cannot be detected reliably by systems trained on static labeled data.

Moreover, most Internet traffic today is encrypted and thus the majority of the IDS solutions have issues with Deep Packet Inspection. In order to effectively develop an IDS solution for modern encrypted network traffic, only payload information is not sufficient for detecting attacks. It is also required to utilize information from other sources like header and flow statistics [4] for accurate detection. Several IDS solutions like Guo et. al. [5], Bhale et. al. [6], Koc et. al. [7], etc. utilizing the flow level information have been developed earlier to resolve this problem.

Earlier approaches like rule-based IDS solutions (i.e. Snort [8], and Bro [9]) consider header fields and payload distribution details only for attack detection, due to which payload-related attacks are not captured. On the other hand, solutions like Perdisci et. al. [10], Ariu et. al. [11], and Thorat et. al. [12] consider only the payload information and do not utilize header or flow level information due to which the flow sequence information is missing and such solutions are unable to detect certain attacks. In other solutions, the packet header and payload are utilized together but they are unable to detect low rate attacks.

To address these limitations, this paper proposes MVNIDS, a Multiview-based Network Intrusion Detection System that employs a self-supervised learning paradigm. The proposed method eliminates the

* Corresponding author.

E-mail addresses: sunitknandi@iitg.ac.in (S.K. Nandi), ritesh.ratti@gmail.com (R. Ratti), ranbir@iitg.ac.in (S.R. Singh), sukumar@iitg.ac.in (S. Nandi).

<https://doi.org/10.1016/j.jisa.2025.104321>

Available online 2 December 2025

2214-2126/© 2025 Elsevier Ltd. All rights are reserved, including those for text and data mining, AI training, and similar technologies.

dependency on manually labeled attack data by leveraging multiple perspectives, or views, of the same network traffic. Specifically, three complementary views are constructed from raw packet capture data, namely, Network View, Flow View, and Image View. Independent autoencoder models are trained for each view using only normal traffic, enabling them to learn compact representations of benign behavior. During inference, reconstruction errors from these view-specific models are combined through a majority-voting mechanism to automatically generate pseudo-labels (“Attack” or “Benign”). These self-generated labels are subsequently used to train a binary classifier capable of accurately predicting the class of unseen traffic instances.

The multiview design serves two key purposes. First, it enhances detection accuracy by integrating diverse information sources: packet headers that capture protocol-level statistics, flow records that capture temporal dependencies, and image representations that encode abstract visual patterns. Second, it empowers the system to detect zero-day and variant attacks. When new attack types exhibit behavioural, temporal, or statistical similarities to known malicious activity, the system can still identify them as anomalies without relying on predefined signatures or exact feature matches. Among these views, the Image View provides a unique advantage by transforming packet data into visual representations that are protocol-agnostic. This allows the system to detect visually similar malicious patterns even when attacks differ in transport mechanism or application context. The Flow View, on the other hand, captures sequence-based temporal dependencies, identifying repetitive or sustained activity across multiple sessions or applications. Together, these views create a comprehensive understanding of network behavior that enables robust, generalized intrusion detection.

In other words, the proposed MVNIDS framework addresses the key limitations of traditional intrusion detection approaches by combining self-supervised learning with a multiview representation strategy mitigating the challenges of labeled data scarcity and class imbalance and extending the system’s capability to detect unknown or evolving attacks in real-world environments. The proposed framework has been evaluated on a recent benchmark dataset, namely, CICIDS2018 [13] on the FTP Bruteforce and UDP DDoS attacks, and its efficacy demonstrated in details. The superiority of the proposed framework is reported by comparison with existing works.

The paper is organized into different sections as follows. In Section 2, we discuss the various solutions proposed on different views derived from network packets for attack detection. Section 3 describes the proposed methodology and algorithm in detail and the components involved. Section 4 presents the implementation of the proposed approach, dataset, & experimentation results. Section 5 brings forward the discussion on generalizability of the proposed method for varying attack types. In Section 6 we provide a conclusion and future work.

2. Background

Network packets consist of two important parts i.e. Network header (describing the protocol header-specific information) and Network payload (consisting of the data bytes transferred). These network packets can be further aggregated based on various parameters, and statistical features like byte per sec, inter-arrival time, sub-flow information, etc can be computed to build the flow level information. Other representations can also be generated similarly with the further transformation of packets to binary formats like Images [14], Text [15], etc. We study various IDS solutions proposed earlier that consider multiple views like network header (Section 2.1), flow (Section 2.2), image (Section 2.3), generated from network packets. A brief description of different methods is presented as follows:

2.1. Network view based solutions

Network view-based solutions consider the information extracted from network headers. Most of the earlier proposed solutions PHAD [16]

and SPADE [17] consider building profiles for network header fields and detecting attack activity by comparing them with the pre-determined thresholds. Another proposed method [18] considers the statistical analysis of packet header data to detect attacks. It is based on analyzing the correlation of destination IP in outgoing traffic and uses wavelet transforms to study the address and port number correlation for different timescales. Most of these proposed solutions are based on outdated datasets and are not relevant to recent attacks.

2.2. Flow view based solutions

The Flow-based solutions proposed earlier consider a supervised learning-based approach like Tomio [19] proposed a multiple flow features-based classification model that considers different views of MAWIFlow dataset [20] like Nigel, Orunada & Viegas features. It consists of two main steps of classification with an autonomous model update, where the classification decision is based on the consensus of random forest models trained for each set of features using majority voting. Other recent flow based solution by Alzubi et. al. [21] and Najafi et. al. [22] utilized sophisticated feature selection to utilize important features to construct various supervised machine learning models.

Other recent Deep CNN based models by Kaur et. al. [23], Wanjau et. al. [24], Kim et. al. [25], Dai et. al. [26] and Kim et. al. [27] utilized the wide set of features to build intrusion detection model that improve the overall detection accuracy but unable to capture the contextual information. To resolve this issue, sequence based model is developed by Hai et. al. [28] that introduces high time complexity due to sequence based approach. Although, the proposed supervised learning based methods provide better results however these methods require frequent data update for new attack detection.

Various unsupervised deep learning-based solutions are also proposed to mitigate the problems of supervised learning-based models. For instance, autoencoder-based solutions proposed by Said et. al. [29], Binbusayyis et. al. [30], Mirsky et. al. [31], Han et. al. [32], Lopes et. al. [33] and similarity learning-based methods proposed by Andresini et. al. [34], Hindy et. al. [35] are discussed earlier that utilize flow level information. The major limitation of most of these deep learning based models is that they suffer from high computation time and complexity.

2.3. Image view based solutions

Most of the other proposed methods also consider representing network packets in binary format (primarily image) and developing the classification model on the represented information. Irina [36] proposed a malware detection method based on binary visualization and self-organizing incremental neural network (SOINN [37]). The algorithm chosen for binary representation uses Hilbert space-filling curves [14] for a better imprint of the image. The proposed method first converts the binary files into an image and then considers 4 parts of an image as the region of interest to construct the histogram with a feature-length of 1024 and utilized the SOINN model for malware detection. The dataset used for experimentation consists of 2K benign files from a trusted source and 2K malicious files collected from the VirusShare website. The detection accuracy of 91.7% and 94.1% is achieved for ransomware attacks in .pdf and .doc files.

Another method for IoT malware traffic detection is proposed by Robert et. al. [38], which considers transforming network traffic into 2D images using the binvis.io [39] utility and classifying the images using the MobileNet model. In the first step network traffic is captured using a sniffer and then to convert the binary file to 2D image data is assumed as a byte string and converted to specific color using an ASCII table. The proposed approach achieved an accuracy of 91.32%. Another similar method proposed by Joseph [40] for IoT network attack detection uses a combination of network traffic profiling and machine learning components. The Network profiling component uses rate metrics for calculating the deviation from normal profile due to attack and

the machine learning component built on MobileNetV3 [41] classification model uses extracted images from packet capture (pcap) files for attack detection. The combination of both systems results in an average accuracy rate per iteration of 99.17 %. Most of these proposed solutions are based on supervised learning mechanisms that require labeled data for training.

2.4. Multi-aspect based solutions

We also study various approaches that consider multiple aspects generated from the same feature set and build supervised learning-based models for detection. For example, the classification method proposed by Ming [42] considers spatial and temporal views together. It consists of 3 layers parallel network structure where the first and third sub-networks consider spatial features using a convolution neural network model (CNN) based model and the second network simultaneously learns spatial and temporal features using the CNN-LSTM network. The proposed network learns the network using early fusion technology to improve the accuracy of network traffic classification. This proposed method is tested on ISCX2012 and CICIDS2017 datasets and provides a classification accuracy of 99.9%. Another solution HCRNNIDS [43] is proposed earlier to initially process the features through CNN and later passed through recurrent neural network (RNN) layers to generate the sequence at each time step. CNN is used to capture the local features while temporal features are captured using RNN. This method is applied to the CIC-IDS2018 dataset and detects attacks with an accuracy of 97.75 %.

The major limitation of the earlier proposed methods is that it considers a single view or set of features at a time and multiple aspects from one type of features are utilized only in the supervised learning-based solution. Our proposed model considers multiple views together to build an IDS solution that is based on a self-supervised learning mechanism and avoids the requirement of labeled data for model training. In the next section, we discuss the proposed methodology in detail.

3. Proposed methodology

The proposed method considers the raw packets from network communication. Various packet sniffing tools like Wireshark, tcpdump, [44] etc. can be used to capture these raw packets from the port in dedicated machine or network switch that is running in promiscuous mode. These raw packets contain communication information among different hosts. For the experiment purpose, we used the raw packets already extracted for specific attacks, but in the real network, a sniffing module is required to capture the packets consecutively with a pre-defined time window length. As shown in the flow diagram (Fig. 1), the extracted network packets are first converted to multiple different views using the subset of packets captured at fixed time window length and later the view-specific autoencoder models are trained and view-specific reconstruction error thresholds are estimated by training the autoencoder models with reconstruction error as a loss function. These trained encoders (part of autoencoder) are further utilized for generating representation from the extracted views and used for producing labeled data with the majority voting on view-specific encoder classification results. This label generation process also handle the scenario for the slight change on the threshold values to build the more robust model that can handle the minimal change in learned threshold as the value of threshold may vary over time due to change in network characteristics. In the final stage of processing, the prepared labeled data is utilized to build the binary classification model that is used to predict the outcome as Benign or Attack.

The proposed solution for the intrusion detection system consists of various modules of processing as shown in the high-level architecture diagram (Fig. 2) for the proposed model training method. The pseudo-code for the proposed model training and inference is represented in

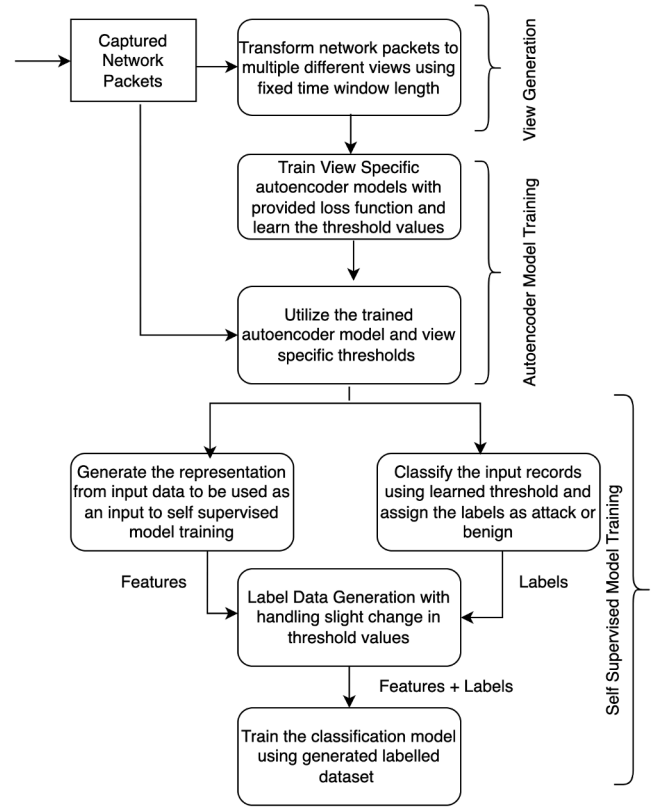


Fig. 1. Flow diagram for multi-view based IDS model training.

Algorithms 1 and 2 respectively. Section 3.1 presents how various views are generated from network packets, Section 3.2 discusses the details on training view specific auto-encoders and Section 3.3 presents the self supervised model training process. Section 3.4 discusses the algorithms for training and inference process in detail.

3.1. View generation

Once the packets are captured from the packet sniffing module, various views are extracted in View generation module. These views exhibit differentiated representations of network packets in multiple forms and constructed views represent supplementary as well as complementary properties intrinsically. For a the pre-defined time window of ΔT , we first extract the batch of network packets based on generated timestamp and use these batches to construct multiple different views as follows:

3.1.1. Network view

The Network View generation extracts protocol header fields from all TCP/IP stack layers (data link through application layer), providing comprehensive visibility across network communications. Features as specified in Table 1 are then aggregated within defined time windows (ΔT) of 1 min, 2 min, 3 min and 4 min to reveal statistical patterns invisible in individual packets. The resulting Network view features include protocol distribution metrics (percentage of TCP, UDP, ICMP traffic), connection establishment patterns (SYN packet rates, connection attempt distributions), fragmentation statistics (fragment counts, fragmentation offsets), and addressing patterns (unique source/destination counts, address entropy measures).

To construct the Network view, we apply specific filtering rules that extract meaningful patterns from the raw header data. For instance, to identify broadcast traffic patterns that might indicate network scanning or ARP poisoning attacks, we filter packets with the destination MAC address ff:ff:ff:ff:ff:ff. Similarly, to calculate incoming traffic volume to

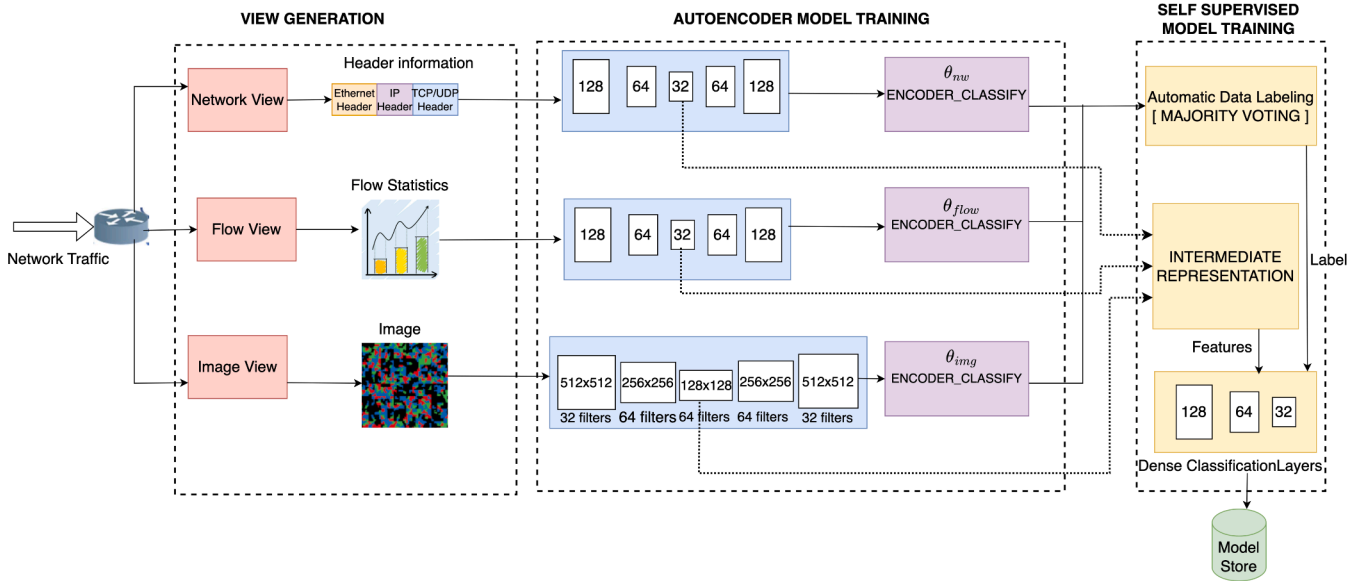


Fig. 2. High level architecture for model training.

Table 1

Network view: Feature information.

S.No.	Feature Category	Feature Description	Count
1	Interface Based	Features related to transferred octets incoming and outgoing, incoming and outgoing unicast and non-unicast packets.	6
2	IP header based	Features related to incoming and outgoing IP datagrams.	2
3	ICMP header based	Features related to number of incoming and outgoing icmp total, echo, destination unreachable messages.	6
4	TCP header based	Features specific to TCP header information like incoming and outgoing segments, connection information.	7
5	UDP header based	Features constructed from UDP header information like input and outgoing datagrams.	3

a potential victim, we filter packets where the destination IP address matches the monitored host's address. These rule-based aggregations are applied systematically across all relevant protocol fields to build a comprehensive network traffic profile.

By focusing on header information rather than payload content, the Network View provides an efficient first-line analysis that is both computationally lightweight and privacy-preserving. This approach allows us to detect attacks that manifest as anomalies in connection patterns, protocol usage, or traffic distribution without the need for deep packet inspection, making it particularly valuable for encrypted traffic analysis where payload examination is not possible.

3.1.2. Flow view

The Flow View transforms raw traffic into bidirectional flow representations using the standard 5-tuple identifier (source IP, destination IP, source port, destination port, protocol). For each flow, we compute various statistical features like volume metrics (byte and packet counts, average sizes), timing characteristics (inter-arrival times, flow duration), flag analysis (distribution of TCP flags throughout the connection lifecycle), throughput patterns (bytes per time unit at different temporal granularities) and other features as specified in Table 2. The features are captured using utilities like Netflow [45] and CICFlowMeter-V3 [46].

These features are calculated for both directions of each flow, then combined to create bidirectional flow profiles. Flow aggregation is further employed to construct time-series representations at various distinct time windows (ΔT) of 1 min, 2 min, 3 min and 4 min.

3.1.3. Image view

The Image View generation methodology transforms abstract network data into visual patterns. This abstract data contains both network header and payload information. The payload information can exist in either non-encrypted or encrypted (with TLS, DTLS, etc.) form. A struc-

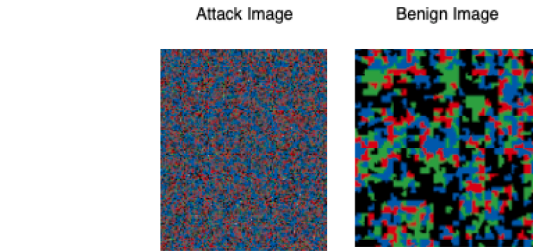


Fig. 3. Image view for benign and attack scenario.

tured image representation is created where header and payload information from captured raw packets is combined to create a 2D representation for each time window ΔT (1 min, 2 min, 3 min and 4 min) using binvis.io [39] utility.

binvis.io [39] internally uses Hilbert Space filling curve [14] for clustering the data which generates a better imprint of an image that provides an added advantage of representation over black and white image. In this algorithm, the image data is divided into multiple bytes and each byte is assigned to a specific color code depending on its ASCII values within the range of ASCII characters as follows:

- blue for printable character
- green for control characters
- red for extended characters
- black for the null character
- white for the non-breaking space

It is also observed that images related to attack consist of dense pixel values with red and blue regions, which is easily distinguishable from normal scenarios where most of the images consist of sparse black regions as represented in Fig. 3. It also conveys that amount of packets

Table 2
Flow view : Feature information [13].

S.No.	Feature Category	Feature Description	Count
1	Packet Transfer rate	Features related to forward and backward packet transfer rate	5
2	Packet Length	Features related to forward and backward packet length information	15
3	Inter-arrival Time	Elapsed time information between flows	14
4	Header information	Features specific to header information like flag count, header length etc.	14
5	Segment information	Features specific to segment size information	4
6	Transfer rate	Features specific to bulk transfer rate	7
7	Subflow related	Features specific to subflow information	7
8	Flow activeness	Statistical Features specific to flow activeness information	8

generated for high rate attacks like DDoS attack is more due to which dense regions are created.

Once the views are generated, the information is represented into numerical features. These numerical features can follow a different range of values. So to avoid model bias, it is required to convert them into a common scale. For this, we used the Normalization technique to transform features in the range of [0,1]. Let us assume the feature values for a specific view is represented in vector $\hat{X}^i$ then the transformed value is X^i is the normalized representation computed through Eq. (1) where, μ is mean and σ is the standard deviation of $\hat{X}^i$. For the Image view, we divide each pixel value by 255 so that values are aligned in the range from [0,1].

$$X^i = \frac{\hat{X}^i - \mu}{\sigma} \quad (1)$$

3.2. Auto-encoder model training

Autoencoder Model Training module first performs the job of training individual view-specific auto-encoder models for normalized view specific features (X^i), and learn the reconstruction error thresholds specific to each view. To build the view-specific auto-encoders on provided data, we assume input data for view is X^i which is represented as a set of N records $x_1, x_2, x_3, \dots, x_N$. Each of these input records (represented with normalized features) is supplied to the Encoder network. The encoder module consists of multiple layers and each layer is associated with multiple processing units. These units are selected based on the type of each view. For instance, Multilayer Perceptron (MLP) units are used for network and flow view (1 dimensional) and CNN units are used for Image view (2 dimensional). The flow of records is processed through each encoder unit. At the last stage of the encoder, an intermediate representation is generated which is provided to the Decoder module. The decoder module consists of the same number of stages as specified in the corresponding encoder layer and tries to reconstruct the supplied information at the subsequent stage of processing. At the final stage, Decoder generates the view-specific output as Y^i which is of the same dimension as the input X^i . The final layer of the decoder generates an output vector and Mean Square Error (MSE) is used to calculate the error between the input and output vector using Eq. (2). This error helps us to quantify the view specific reconstruction error threshold θ^i for normal traffic which is based on the mean of the error values as shown in Eq. (3). Once the threshold is determined, incoming flow is identified to be an attack specifically if the reconstruction error is larger than threshold θ^i . Let D is the feature dimension for view X^i and N is the total number of training records then the error value $Error^i$ and threshold θ^i is estimated as

$$Error^i = \frac{1}{D} \sum_{d=1}^D |X^i(d) - Y^i(d)|^2 \quad (2)$$

$$\theta^i = \frac{1}{N} \sum_{n=1}^N Error^i(n) \quad (3)$$

After this stage, the set of various thresholds ($\theta^1, \theta^2, \dots, \theta^n$) are learned for each specific view, which is used to classify the input records. These

individual thresholds are used to label the individual records as Benign or Attack in the next stage.

3.3. Self-supervised model training

Automatic data labelling is the next step that delivers self-supervision for the proposed scheme. It is used to generate the attack or benign labels from the pre-trained view-specific autoencoder models. This stage utilizes the view-specific thresholds learned in the previous stage to compute the label for an outcome of each record when passed through a learned autoencoder. For a given record, multiple views are processed through each view-specific encoder separately to identify the attack activity. Now the final label specific to the network event is assigned as the majority vote of the individual encoders. As shown in Fig. 4 majority of outcome from individual view-based autoencoder is assigned as a final label (anomaly as True or False) for further training. In this example, loss values and thresholds for network, flow and image view are defined as (loss_nw, loss_flow, loss_img) and (threshold_nw, threshold_flow, threshold_img) respectively. In the proposed mechanism majority voting from 3 (odd number of views) autoencoders is considered as a majority. This stage also handles the slight change ($\theta^i \pm \delta$) in the values of learned view specific thresholds by estimating the slightly modified threshold and regenerating the labels. By considering the slight change in threshold, the trained model becomes more robust towards the minimal changes in network characteristics and provide better results. During inference, this model can directly be used to classify an attack instance, instead of considering the majority voting from the autoencoders.

For the majority voting, Let $C_1(X), C_2(X), \dots, C_n(X)$ be the predicted classes from n different view-based encoder models for an input vector X then, final label $C_{major}(X)$ can be computed with Eq. (4).

$$C_{major}(X) = mode(C_1(X), C_2(X), \dots, C_n(X)) \quad (4)$$

The data prepared using self-supervision is used now to build the binary classifier. In this step, we utilize the intermediate representations from view specific autoencoders and used as an input to binary classifier. View-specific encoders are first used to encode the incoming view and generate representation. These representations are further concatenated together and supplied to classifier consisting of fully connected layers with softmax layer as an output. This binary classifier consists of 3 dense layers with 128,64,32 neurons and utilize binary cross entropy loss function for training the model. More details on the learning structure of the network can be found in Table 3.

3.4. Algorithm

The proposed algorithm involves multiple stages of processing as described earlier in Fig. 2. These stages play a vital role in the overall process. The Pseudo-code for the proposed model training and inference is represented in Algorithms 1 and 2 respectively.

3.4.1. Model training

The Algorithm 1 presents the model training process in detail. For the pre-selected time window ΔT and raw packets P , different views

	loss_nw	threshold_nw	anomaly_nw	loss_flow	threshold_flow	anomaly_flow	loss_img	threshold_img	anomaly_img	anomaly
0	0.389049	0.220287	True	0.287950	0.221312	True	0.101505	0.096446	True	True
1	0.390333	0.220287	True	0.769167	0.221312	True	0.101798	0.096446	True	True
2	0.315964	0.220287	True	0.054242	0.221312	False	0.086322	0.096446	False	False
3	0.047842	0.220287	False	0.052423	0.221312	False	0.089829	0.096446	False	False

Fig. 4. Sample of automatic data labeling using majority voting.

Table 3

Model configuration for binary classifier.

Parameter	Values
Additional dense Layers	3
Concatenated Encoder Representation Length	96
Layer Structure	128, 64, 32
Optimizer	Adam
Learning rate	1e-4
Loss function	Binary cross entropy
Batch size	16
Number of Epochs	50

Algorithm 1: Model training.

Input: Time window ΔT , δ , and collection of raw packets P
Output: Classification Model M , and Autoencoder models $A^{network}$, A^{flow} , A^{image}

- 1: $X^{network}, X^{flow}, X^{image} \leftarrow VIEW_GENERATION(P, \Delta T)$
- 2: $X \leftarrow X^{network}, X^{flow}, X^{image}$
- 3: **for** $X^i \in X$ **do**
- 4: $A^i, \theta^i \leftarrow AUTOENCODER_MODEL(X^i)$
- 5: **end for**
- 6: Initialize label array as L
- 7: **for** $P^j \in P$ **do**
- 8: Initialize class array as C
- 9: **for** $X^i \in X$ **do**
- 10: $C_j^i \leftarrow ENCODER_CLASSIFY(\theta^i \pm \delta, P^j)$
- 11: Add C_j^i to C
- 12: **end for**
- 13: $L^j \leftarrow MAJORITY_VOTING(C)$
- 14: **end for**
- 15: Model $M \leftarrow CLASSIFICATION_MODEL(P, L)$

$X^{network}$, X^{flow} , X^{image} are generated at an initial stage. The view generation functionality is represented as $VIEW_GENERATION()$ which is used to create multiple views as per sub-Section 3.1. Later, for each of the views, separate autoencoder models are trained, and view-specific thresholds θ^i are determined. The autoencoder training process is defined as function $AUTOENCODER_MODEL()$ that trains the model on supplied view X^i . Now with each autoencoder, an input record is predicted as Normal or Benign depending on comparing calculated loss with learned view-specific threshold. The process of prediction is called $ENCODER_CLASSIFY()$ that takes the learned thresholds and the variation factor δ as input. Additional records are generated by including the slight change (δ) in threshold values. Further, the majority voting is applied on predicted class metrics, and the final label is identified for an input record. The associated function $MAJORITY_VOTING()$ considers the outcome from each view-specific autoencoder and selects the majority as a final outcome. This information is generated for all input records. In the final stage, the binary classifier M is trained with labeled data and stored for the inference process. The classification model is trained in function $CLASSIFICATION_MODEL()$ by using the final labels generated at the previous step and using the specified configura-

tion in Table 3. The whole training process is explained below in the Algorithm 1.

Complexity Analysis: The overall complexity of the training process depends upon the associated functions. The $VIEW_GENERATION()$ function considers calculating the aggregated features for the set of packets in the specified time window. The specified views can be computed in parallel with linear complexity therefore for V such views and F features, the total complexity is $O(V * F)$. The complexity of training an autoencoder model with a constant number of layers and with N computation units is around $O(N^2)$ and multiple view-specific autoencoders can be trained in parallel too. Function $ENCODER_CLASSIFY()$ also takes $O(N^2)$ time for encoding the view-specific information. The final classifier trained is a multi-layer perceptron and its complexity is also $O(n^2)$ for n computation units. Hence the overall complexity for the model training process for a particular batch is around $O(V * F) + O(N^2) + O(n^2)$.

3.4.2. Model inference

The Algorithm 2 represents the execution of the proposed scheme during inference where the trained models are first loaded from persistent storage. It utilizes the view-specific autoencoder models for encoding purpose and uses the classification model directly to classify the generated view-specific representations as Benign or Attack. In the first step, the packets captured at a pre-defined time window ΔT are used to generate the multiple views iteratively by using the function $VIEW_GENERATION()$. Later these views are processed from the respective encoders to generate the representations (i.e. $R^{network}$, R^{flow} , R^{image}). This encoding is represented as a function $ENCODE()$ that takes a trained autoencoder and view as an input and generates the view-specific representation. These representations are further classified from the trained self-supervised model to predict a Benign or an Attack scenario and are represented as a function $CLASSIFY()$.

Complexity Analysis: The overall complexity of the proposed model inference algorithm depends on the functions $VIEW_GENERATION()$, $ENCODE()$, and $CLASSIFY()$. For an incoming set of packets captured at time window ΔT , V views are generated for F features so the complexity is around $O(V * F)$. The function $ENCODE()$ is used to encode the view information through autoencoder which is $O(N^2)$ for N computation units. and $CLASSIFY()$ function generally takes $O(n^2)$ for n computation units. Therefore the overall complexity of algorithm is $O(V * F) + O(N^2) + O(n^2)$.

Algorithm 2: Model inference.

Input: Model M , Time window ΔT , $A^{network}$, A^{flow} , A^{image} and collection of raw packets P
Output: Benign or Attack

- 1: Load models M , $A^{network}$, A^{flow} , A^{image}
- 2: $X^{network}, X^{flow}, X^{image} \leftarrow VIEW_GENERATION(P, \Delta T)$
- 3: $R^{network} \leftarrow ENCODE(A^{network}, X^{network})$
- 4: $R^{flow} \leftarrow ENCODE(A^{flow}, X^{flow})$
- 5: $R^{image} \leftarrow ENCODE(A^{image}, X^{image})$
- 6: Benign or Attack $\leftarrow CLASSIFY(M, R^{network}, R^{flow}, R^{image})$

Table 4

Model configuration for network and flow view.
The **f** indicates input feature vector.

Parameter	Values
Total Layers	9
Encoder structure	f,128, 64, 32
Compressed output length	16
Decoder structure	32, 64, 128, f
Optimizer	Adam
Learning rate	0.0001
Loss function	Mean squared error
Batch size	16
Number of Epochs	20

Table 5

Model configuration for image view.

Parameter	Values
Total Layers	6
Input Dimension	512x512x3
Encoder structure	512x512x32, 256x256x64
Compressed dimension	128x128x64
Decoder structure	256x256x64, 512x512x32
Optimizer	Adam
Learning rate	0.0001
Loss function	Mean squared error
Batch size	16
Number of Epochs	20

4. Experimentation results

4.1. Dataset

To evaluate the performance of the proposed method, we consider CICIDS2018 dataset [13] which is recent and publicly available dataset developed at Canadian Institute for Cybersecurity (CIC) where FTP-Patator and SSH-Patator tools [47] are used for performing brute force attacks and the LOIC, HOIC tools are used for performing DDoS attacks. To prepare the dataset, network flow records are generated on traffic data by using CICFlowMeter-V3 tool [46] and in total 7 different types of attacks are captured at different dates. It also contains data in raw network packet files also which is only used for experimentation purpose. For the purpose of our evaluation, we consider raw packets for two representative attacks, namely, FTP BruteForce and UDP DDoS along with Benign traffic. We utilized raw packet data specifically for the victim host (IP address: 172.31.69.25) and constructed various views from it to build the model. The selected pcap files consist of 4718327 number of packets in total for FTP brute force-specific attacks performed on Wed-14-02-2018 date and a total of 91,666,016 packets for UDP DDoS-specific attacks that was preformed on Tue-20-02-2018 date.

4.2. Implementation

For our implementation, we extracted data for the Benign scenario specific raw packets by utilizing the timestamp information when any attack has not been performed and generated various .pcap file chunks using editcap utility [48] with the defined time window length from the entire packet capture from the dataset. In various experiments, we increase the time window size (from 1 min. upto 4 min.) and analyze how the model performs with the increment as the time window affects the amount of data generated after aggregation. These binary files (.pcap) are used further to generate multiple views, where binvis.io [39] is used for image view generation, CICFlowMeter-v3 [46] is used for preparing flow view, and network view is generated using our handcrafted code. Further, view-specific autoencoder models are trained and view-specific thresholds are identified for each view individually. The complete implementation is performed on Mac OS with 6-core Intel Core-i7 processor with 16 GB RAM and utilized the TensorFlow Python pack-

age. The configuration for autoencoder networks for different views are used as shown in Tables 4 and 5. We validated the model performance in terms of classification metrics considering the timestamps specific to Attack or Benign class from the dataset description. We also evaluated the run time performance of the proposed model in terms of the number of flows processed per second in run time. We used $\delta = 0.10$ as a value to consider the change in threshold parameters while training the self-supervised model. This value is selected by observing the standard deviation in the estimated thresholds after multiple runs. If the threshold change factor (δ) is very high the false positives are reduced, that leads to higher precision.

4.3. Results and analysis

We evaluated the detection performance of the proposed model in terms of precision, recall, F1-score, and accuracy for various time windows (1 min, 2 min, 3 min, and 4 min). In general window size refers to the time frame or number of data points considered for analysis at any given moment. The intuition behind selecting specific window sizes involves balancing between detecting short-term anomalies and capturing long-term patterns. Too small windows might capture too much noise, leading to false positives, while too large windows may smooth out significant anomalies, leading to false negatives. The selection of window size is based on these intuitions. We have considered the maximum window size of 4 min. to capture the attacks activity within this duration.

To estimate the prediction performance, we hold out 10% of normal data as testing along with the attack dataset, and estimated the evaluation metrics between *Benign* or *Attack* classes. We experimented the models with different time window lengths and report the results for each time window since different values exhibits different performance figures as mentioned in Tables 6 and 7 for FTP Brute force and UDP DDoS attacks respectively. We present results for the two proposed methods with majority voting and with self supervised learning separately, defined as Proposed M.V. and Proposed S.S. respectively. From the results, it is evident that the proposed model performance varied with time window length. It is analyzed from the performance results that the proposed model outperforms every other individual view in terms of precision and accuracy for most of the time window values. The best F1-score and accuracy for FTP Brute force attack is attained as 98.3% and 98.5% respectively for a time window of 1 min. The best F1-score and accuracy results for UDP DDoS attack is attained as 98.1% and 97.2% respectively for time window of 3 min.

It is also observed that the proposed majority voting based method always performs better than the flow view and network view separately for both the attacks. The proposed self-supervised learning-based model also outperforms the majority voting in terms of precision, F1-score and accuracy for specific time window length. It is estimated that recall values are lower for the proposed self supervised learning based model due to introduction of the slight change in threshold and generating data inherently which consists of more records for normal scenario. However with the proposed self supervised learning based scheme, model is able to capture the slight change in threshold values and handle the attack scenarios better.

It is also observed from view-specific results, that flow view and image view are able to well capture the FTP Brute Attacks because it is based on a connection-oriented protocol (i.e. TCP) and the performance results for network view are lower. On the other hand for UDP DDoS attack Network view performs better than flow view and image view because it is based on connection-less protocol (i.e. UDP) and network view is the best representative to explain the attack. We also analyzed that image view performance (F1-score and accuracy) diminishes for larger window sizes.

4.3.1. Model convergence

To assess the learning stability and optimization behavior of the proposed model, an epoch-wise convergence analysis is conducted on the

Table 6

Performance comparison of different auto-encoder models for FTP Bruteforce attack.

Window Size ΔT	Model	Precision%	Recall%	F1Score%	Accuracy%
1 min	Network View	93.0	100	96.3	94.1
	Flow View	94.9	100	97.3	95.8
	Image View	97.3	100	98.2	97.9
	Proposed M.V.	96.8	99.4	98.1	97.0
	Proposed S.S.	98.7	98.1	98.3	98.5
2 min	Network View	89.5	100	94.4	90.9
	Flow View	93.0	100	96.4	94.2
	Image View	95.6	100	97.7	95.9
	Proposed M.V.	93.9	98.9	96.4	94.2
	Proposed S.S.	98.2	96.0	95.9	96.0
3 min	Network View	92.4	98.3	95.3	92.5
	Flow View	91.0	98.3	94.5	91.2
	Image View	95.1	100	97.5	95.9
	Proposed M.V.	93.9	100	96.8	95.0
	Proposed S.S.	98.3	90.1	94.0	94.3
4 min	Network View	90.3	100	94.9	91.8
	Flow View	97.8	97.8	97.8	96.7
	Image View	97.6	100	97.6	96.0
	Proposed M.V.	96.0	100	97.9	95.7
	Proposed S.S.	98.1	93.1	95.5	96.0

Table 7

Performance comparison of different auto-encoder models for UDP DDoS attack.

Window Size ΔT	Model	Precision%	Recall%	F1Score%	Accuracy%
1 min	Network View	90.0	100	94.1	90.2
	Flow View	90.0	80	84.9	79.8
	Image View	93.0	97.5	95.2	93.6
	Proposed M.V.	93.0	100	96.3	94.6
	Proposed S.S.	98.5	77.12	86.5	90.8
2 min	Network View	87.2	100	93.1	89.6
	Flow View	91.6	80.4	85.7	81.0
	Image View	89.3	100	94.3	92.1
	Proposed M.V.	93.0	100	94.4	94.7
	Proposed S.S.	97.9	91.1	95.4	95.5
3 min	Network View	96.2	100	98.1	97.2
	Flow View	91.6	84.6	87.9	83.7
	Image View	87.0	100	93.1	90.4
	Proposed M.V.	96.2	100	98.1	97.2
	Proposed S.S.	100	81.0	90.0	93.2
4 min	Network View	84.3	100	91.3	86.2
	Flow View	90.0	95.2	93.0	89.6
	Image View	71.4	100	83.3	74.2
	Proposed M.V.	90.9	100	95.2	92.8
	Proposed S.S.	100	68.0	81.0	87.2

autoencoder component during training, as illustrated in Fig. 5a. The figure presents the epoch-wise progression of training and validation loss over 20 epochs. The training loss begins at approximately 0.72 and steadily decreases to 0.38, while the validation loss exhibits a parallel downward trend from 0.25 to 0.13, indicating strong generalization and consistent learning throughout the process.

The convergence pattern reveals four distinct phases:

- **Rapid initial improvement (Epochs 1-3):** The model quickly learns fundamental data representations, resulting in a sharp decline in training loss.
- **Intermediate fluctuations (Epochs 3-8):** Minor oscillations appear as the model explores a complex error landscape while refining feature reconstruction.
- **Consistent convergence (Epochs 8-14):** Both loss curves show a smooth, steady decline, signifying stable learning and progressive optimization of latent feature mappings.

- **Final stabilization (Epochs 15-20):** The model approaches convergence with minor oscillations, reflecting fine-tuning of parameters without any indication of overfitting.

The maintained gap of approximately 0.25 between training and validation losses throughout training demonstrates effective regularization and robustness. The absence of validation loss spikes confirms model stability, while the final low validation loss of 0.13 underscores the autoencoder's ability to generalize effectively to unseen intrusion patterns. These results collectively highlight the model's suitability for real-world deployment in network intrusion detection systems requiring both accuracy and reliability.

4.3.2. Time window length vs processing rate

Further, this section also investigates the processing rate in terms of records per second processed by the trained model against the provided time window length (in minutes), as shown in Fig. 5b. The figure denotes that the processing rate is independent of selected time window as minimal change is identified on increasing time window length. This is due to non-contextual nature of the proposed scheme that process available data for every time window independently. However, the outcome of IDS is delayed due to higher time window length as IDS system requires to wait for processing till the data is available.

4.3.3. Comparison with existing methods

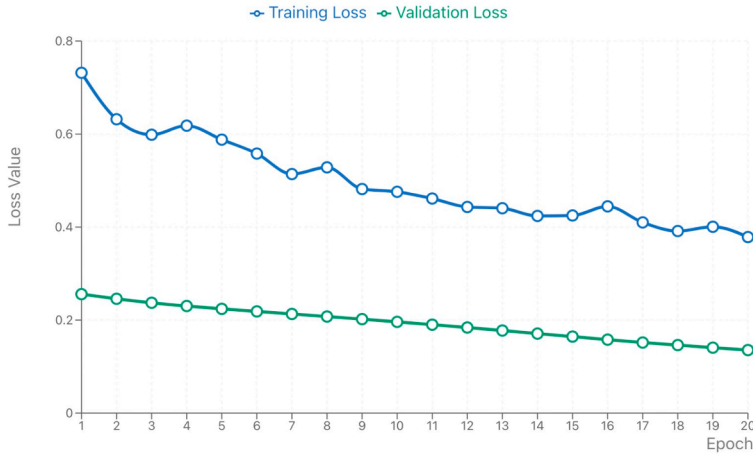
In this subsection, we first provide the overall comparison of the proposed framework with several recent methods build on CICIDS2018 dataset for attack detection. The comparison is based on various properties like utilized feature set, underlying approach, support for zero day attack and overall complexity described in Table 8.

We later presented the performance comparison with existing methods used for FTP brute force attack and UDP DDoS attack detection respectively that are implemented using CICIDS2018 dataset. The performance comparison results are presented in Table 9 for both the attacks. From the results it is evident that the proposed model outperforms most of supervised learning based methods except Alzubi et. al. [21] and Najafi et.al. [22] for FTP Brute Force attack and Lopes et. al. [33] for UDP DDoS attacks. We can analyze that Alzubi et. al. [21] and Najafi et.al. [22] provides better classification metrics for FTP Brute Force attack detection, but being supervised learning based model extra efforts on data labeling is required due to which it is unable to detect zero-day attacks. Also Najafi et.al. [22] suffers from high complexity for ensemble based model training on selected features and extra processing overhead on feature selection and discretization. On the other hand, Lopes et. al. [33] offers better performance compared to our proposed method for UDP DDoS attack, but suffers from the requirement of labelled data for training and involves complex multi-stage model training and inference process. In comparison to specified recent studies, the proposed method delivers comparable performance and is well suitable for detecting new attacks with less complexity and can be used in online detection of attacks as it processes the data in unsupervised manner for specified time window.

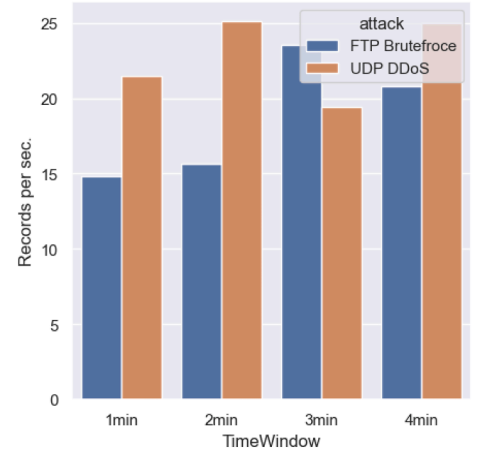
5. Discussion on generalizability

The proposed multiview self-supervised model IDS approach evaluated on the FTP Brute Force and UDP DDoS attacks for the CICIDS2018 dataset demonstrates strong potential for generalizability across diverse intrusion types. These two attacks have different fundamentals: FTP Brute Force aims to compromise authentication through repeated credential attempts, while UDP DDoS targets service availability through traffic flooding and resource exhaustion. The consistent detection performance across such contrasting behaviors implies that the proposed model detects core malicious patterns rather than attack-specific signatures.

SSH Brute Force, RDP Brute Force, and Web Application login abuse attacks share similar behaviours with FTP Brute Force, particularly in



(a) Training and validation loss curves of the autoencoder-based MVNIDS model.



(b) Time window vs Processed records per sec.

Fig. 5. Performance analysis of the MVNIDS model: (a) Model convergence (b) Processing Rate (a) Training and validation loss curves of the autoencoder-based MVNIDS model. (b) Time window vs Processed records per sec.

Table 8

Overall comparison with recent proposed methods on CICIDS2018 dataset.

Method	Approach	Feature Set	Zero-day attack support	Complexity
Alzubi et. al. [21]	Supervised learning with Advanced feature selection.	From 67 Flow level features, group of 10,6,7 features after feature reduction are used.	Unable to identify zero-day attack as re-training is required.	Slower model training due to added complexity of feature selection, results are also highly sensitive to selected features. It uses binary classification model only and ignore attack specific results that can vary due to class imbalance.
Najafi et. al. [22]	Supervised learning with feature selection and discretization.	From a pool of 67 flow level features, group of 5,9 and 10 features are selected after feature selection.	Unable to identify zero day attacks as complex feature engineering is required with change in attack behaviour.	High complexity for ensemble based model training on selected features. Extra processing overhead on feature selection and discretization introduce added complexity which is unsuitable for online IDS execution.
Kaur et. al. [23]	Supervised method - CNN based classification model.	Total 64 flow level features after selection.	Unable to identify zero day attacks.	More complex due to training of Deep-CNN based models and slow inference at run time.
Wanjau et. al. [24]	Supervised learning - Deep CNN based model.	78 Flow level features with PCA based dimensionality reduction.	Require retraining and data preparation for detection of new attacks.	High training complexity due to Deep CNN Model training and slower inference.
Hai et. al. [28]	Supervised Learning - Sequence based models like RNN, LSTM, GRU.	70 flow level features are utilized.	Unable to detect zero day attacks, require retraining and data preparation.	High complexity due to sequence model training and slow inference speed due to sequence prediction.
Kim et. al. [25]	Supervised method - Deep CNN based classification model.	78 numerical Flow level features are used.	Required retraining and data preparation for new attack detection.	Full training of Deep CNN model is highly complex.
Dai et. al. [26]	Supervised - Deep CNN and sparse autoencoder based model.	All flow level features are utilized.	Framework can not identify zero day attacks.	Model training and inference is complex as it involves combination of Deep-CNN and intermediate sparse autoencoders.
Lopes et. al. [33]	Semi-Supervised learning - Denoising Autoencoder with Deep neural network classifier.	All flow level features are utilized.	System can minimally identify zero day attacks.	Complex model training and inference due to multistage execution.
Proposed M.V.	Unsupervised - Majority Voting based	Extensive set of Network (24 features), Flow level (74 features) and Image view are utilized.	Framework can identify zero day attacks.	Medium complexity due to inference from multiple auto-encoders and execution of majority voting process. It can be executed in an online manner.
Proposed S.S.	Self-supervised Learning	Extensive set of Network level (24 features), Flow level (74 features) and Image view are utilized.	Framework can identify zero day attacks	Less run time complexity due to classification model based prediction on constructed views. It requires training classification model with auto-generated labels and can run in online manner.

Table 9

Performance comparison with recent proposed methods on CICIDS2018 datasets.

(a) FTP BruteForce Attack				
Method	Prec.%	Rec.%	F1 %	Acc.%
Alzubi [21]	99.8	99.0	99.4	99.0
Najafi [22]	99.7	98.8	99.3	98.8
Kaur [23]	82.8	42.6	56.3	42.6
Wanjau [24]	92.5	97.8	91.8	94.3
Hai [28]	95.7	95.4	97.5	–
Kim [25]	–	–	–	98.0
Proposed M.V.	96.8	99.4	98.1	97.0
Proposed S.S.	98.7	98.1	98.3	98.5

(b) UDP DDoS Attack				
Method	Prec.%	Rec.%	F1 %	Acc.%
Kim [27]	–	–	–	95
Dai [26]	88.0	100	94.11	93
Lopes [33]	95.6	98.8	97.7	–
Proposed M.V.	93.0	100	94.4	94.7
Proposed S.S.	97.9	91.1	95.4	95.5

terms of repetitive authentication attempts and persistence within distinct protocol contexts. Therefore, the multiview representation capable of identifying FTP-based brute force attacks can be naturally extended to credential stuffing or distributed password guessing attacks, where packet timing and protocol variations differ but the underlying access attempt behaviours remain consistent.

Similarly, the ability to detect UDP-based DDoS attacks provides extensibility to other volumetric and application-layer flooding techniques, including TCP SYN Floods, ICMP Floods, reflection attacks and HTTP-based DoS/DDoS. These attacks produce distinctive flow dynamics and image view representations that deviate markedly from normal traffic patterns, which the proposed multiview encoding captures effectively.

A principal advantage of the multiview design lies in the image view, which abstracts network activity into visual patterns that are mostly protocol-agnostic. This enables the detection of visually similar malicious behaviors even when underlying transport mechanisms differ. Additionally, the flow view encapsulates temporal dependencies, allowing the model to recognize repetitive or sustained activity across various network services. Overall, the proposed multiview methodology demonstrates an inherent generalization capability by focusing on fundamental behavioural characteristics such as repetition, persistence, and deviation from typical flow distributions rather than specific implementation details or protocols. This adaptability positions MVNIDS as a promising framework for detecting both known and emerging network attacks beyond the FTP Brute Force and UDP DDoS categories.

6. Conclusion

We proposed MVNIDS, a Multiview-based Network Intrusion Detection System that leverages a self-supervised learning approach to overcome the limitations of traditional signature-based and supervised IDS methods. By independently training autoencoders on Network, Flow, and Image views derived from raw packet data and combining their outputs through a majority-voting mechanism, MVNIDS effectively generates its own labels and learns robust behavioral representations without manual annotation. Experimental evaluation on the CICIDS2018 dataset for FTP BruteForce and UDP DoS attacks demonstrates that the proposed framework achieves higher accuracy, precision, and F1-scores than most view-specific or supervised baselines, while maintaining reduced computational complexity to be used as an online IDS. The system's multiview design enhances both accuracy and adaptability, enabling detection of zero-day and evolving attacks through protocol-agnostic and temporally aware representations

CRedit authorship contribution statement

Sunit Kumar Nandi: Writing – original draft, Software, Methodology, Investigation; **Ritesh Ratti:** Writing – original draft, Formal analysis, Data curation, Conceptualization; **Sanasam Ranbir Singh:** Writing – review & editing, Visualization, Validation, Supervision; **Sukumar Nandi:** Writing – review & editing, Resources, Project administration, Conceptualization.

Data availability

Research uses publicly available CICIDS2018 dataset. URL: <https://www.unb.ca/cic/datasets/ids-2018.html>

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

References

- [1] Sauter M. LOIC will tear us apart: the impact of tool design and media portrayals in the success of activist DDOS attacks. *Am Behav Sci* 2013;57(7):983–1007.
- [2] Yaltirakli G. Slowloris. 2015. <https://github.com/gkbrk/slowloris>
- [3] Lyon GF. Nmap network scanning: the official Nmap project guide to network discovery and security scanning. Insecure. Com LLC (US); 2008.
- [4] Papadogiannaki E, Tsirantonakis G, Ioannidis S. Network intrusion detection in encrypted traffic. In: 2022 IEEE Conference on dependable and secure computing (DSC). IEEE; 2022, p. 1–8.
- [5] Guo C, Ping Y, Liu N, Luo S-S. A two-level hybrid approach for intrusion detection. *Neurocomputing* 2016;214:391–400.
- [6] Bhale P, Chowdhury DR, Biswas S, Nandi S. OPTIMIST: lightweight and transparent IDS with optimum placement strategy to mitigate mixed-rate DDoS attacks in IoT networks. *IEEE Internet Things J* 2023;10(10):8357–70.
- [7] Koc L, Mazzuchi TA, Sarkani S. A network intrusion detection system based on a hidden naive bayes multiclass classifier. *Expert Syst Appl* 2012;39(18):13492–500.
- [8] Roesch M. Snort: lightweight intrusion detection for networks. In: LISA XIII. 1999, p. 229–38.
- [9] Paxson V. Bro: a system for detecting network intruders in real-time. *Comput Netw* 1999;31(23–24):2435–63.
- [10] Perdisci R, Ariu D, Fogla P, Giacinto G, Lee W. McPAD: a multiple classifier system for accurate payload-based anomaly detection. *Comput Netw* 2009;53(6):864–81.
- [11] Ariu D, Tronci R, Giacinto G. HMMPayL: an intrusion detection system based on hidden Markov models. *comput Secur* 2011;30(4):221–41.
- [12] Thorat SA, Khandelwal AK, Bruhadeshwar B, Kishore K. Payload content based network anomaly detection. In: 2008 First international conference on the applications of digital information and web technologies (ICADIWT). IEEE; 2008, p. 127–32.
- [13] Sharafaldin I, Lashkari AH, Ghorbani AA. Toward generating a new intrusion detection dataset and intrusion traffic characterization. In: 4th international conference on information systems security and privacy. 2018, p. 108–16.
- [14] Jagadish HV. Analysis of the hilbert curve for representing two-dimensional space. *Inf Process Lett* 1997;62(1):17–22.
- [15] Min E, Long J, Liu Q, Cui J, Chen W. TR-IDS: anomaly-based intrusion detection through text-convolutional neural network and random forest. *Secur Commun Netw* 2018;2018(1):4943509.
- [16] Chan PK, Mahoney MV. PHAD: packet header anomaly detection for identifying hostile network traffic. Florida Inst Technol Tech Report CS-2001-04 2001. https://repository.fit.edu/ces_faculty/130/
- [17] Staniford S, Hoagland JA, McAlerney JM. Practical automated detection of stealthy portscans. *J Comput Secur* 2002;10(1–2):105–36.
- [18] Kim SS, Reddy ALN. Statistical techniques for detecting traffic anomalies through packet header data. *IEEE/ACM Trans Netw* 2008;16(3):562–75.
- [19] Tomio RL, Viegas EK, Santin AO, dos Santos RR. A multi-view intrusion detection model for reliable and autonomous model updates. In: IEEE International conference on communications. 2021, p. 1–6.
- [20] Fontugne R, Borgnat P, Abry P, Fukuda K. Mawilab: combining diverse anomaly detectors for automated anomaly labeling and performance benchmarking. In: CoNEXT '10: proceedings of the 6th international Conference. 2010, p. 1–12.
- [21] Alzubi QM, Makhadmeh SN, Sanjalawe Y. Optimizing intrusion detection: advanced feature selection and machine learning techniques using the CSE-CIC-IDS2018 dataset. *J Adv Inform Technol* 2025;16(3):20.
- [22] Najafi Mohsenabad H, Tut MA. Optimizing cybersecurity attack detection in computer networks: a comparative analysis of bio-inspired optimization algorithms using the CSE-CIC-IDS 2018 dataset. *Appl Sci* 2024;14(3):1044.
- [23] Kaur G, Lashkari AH, Rahali A. Intrusion traffic detection and characterization using deep image learning. In: 2020 IEEE Intl conf on dependable, autonomic and secure computing, intl conf on pervasive intelligence and computing, intl conf on cloud and big data computing, intl conf on cyber science and technology congress (DASC/pi-com/CBDCom/cyberscience). IEEE; 2020, p. 55–62.

- [24] Wanjau SK, Wambugu GM, Kamau GN. SSH-brute force attack detection model based on deep learning. *Int J Comput Appl Technol Res* 2021;10(1):42–50.
- [25] Kim J, Shin Y, Choi E. An intrusion detection model based on a convolutional neural network. *J Multimed Inform Syst* 2019;6(4):165–72.
- [26] Dai Q-y, Zhang B, Dong S-q. A DDoS-attack detection method oriented to the blockchain network layer. *Secur Commun Netw* 2022;2022(1):5692820.
- [27] Kim J, Kim J, Kim H, Shim M, Choi E. CNN-based network intrusion detection against denial-of-service attacks. *Electronics* 2020;9(6):916.
- [28] Hai TH, Nam LH. A practical comparison of deep learning methods for network intrusion detection. In: 2021 International conference on electrical, communication, and computer engineering (ICECCE). IEEE; 2021, p. 1–6.
- [29] Said Elsayed M, Le-Khac N-A, Dev S, Jurcut AD. Network anomaly detection using LSTM based autoencoder. In: Proceedings of the 16th ACM symposium on QoS and security for wireless and mobile networks. 2020, p. 37–45.
- [30] Binbusayyis A, Vaiyapuri T. Unsupervised deep learning approach for network intrusion detection combining convolutional autoencoder and one-class SVM. *Appl Intell* 2021;51(10):7094–108.
- [31] Mirsky Y, Doitshman T, Elovici Y, Shabtai A. Kitsune: an ensemble of autoencoders for online network intrusion detection. (2018) arXiv preprint arXiv:1802.09089.
- [32] Han X, Liu Y, Zhang Z, Lü X, Li Y. Sparse auto-encoder combined with kernel for network attack detection. *Comput Commun* 2021;173:14–20.
- [33] Lopes IO, Zou D, Abdulqadder IH, Ruambo FA, Yuan B, Jin H. Effective network intrusion detection via representation learning: a denoising autoencoder approach. *Comput Commun* 2022;194:55–65.
- [34] Andresini G, Appice A, Malerba D. Autoencoder-based deep metric learning for network intrusion detection. *Inf Sci* 2021;569(1):706–27.
- [35] Hindy H, Tachtatzis C, Atkinson R, Brosset D, Bures M, Andonovic I, et al. Leveraging siamese networks for one-shot intrusion detection model. (2020) arXiv preprint arXiv:2006.15343.
- [36] Baptista I, Shiaeles S, Kolokotronis N. A novel malware detection system based on machine learning and binary visualization. In: 2019 IEEE International conference on communications workshops (ICC workshops). IEEE; 2019, p. 1–6.
- [37] Shen F, Hasegawa O. Self-organizing incremental neural network and its application. In: Artificial neural networks–ICANN 2010: 20th international conference, thessaloniki, greece, september 15–18, 2010, proceedings, part III 20. Springer; 2010, p. 535–40.
- [38] Shire R, Shiaeles S, Bendiab K, Ghita B, Kolokotronis N. Malware squid: a novel iot malware traffic analysis framework using convolutional neural network and binary visualisation. (2021) arXiv preprint arXiv:2109.03375.
- [39] Cortesi A. binvis. io: Visual analysis of binary files. 2016. <https://binvis.io/>.
- [40] Rose JR, Swann M, Bendiab G, Shiaeles S, Kolokotronis N. Intrusion detection using network traffic profiling and machine learning for iot. In: 2021 IEEE 7th international conference on network softwarization (NetSoft). IEEE; 2021, p. 409–15.
- [41] Howard A, Sandler M, Chu G, Chen L-C, Chen B, Tan M, et al. Searching for mobilenetv3. In: Proceedings of the IEEE/CVF international conference on computer vision. 2019, p. 1314–24.
- [42] Li M, Han D, Yin X, Liu H, Li D. Design and implementation of an anomaly network traffic detection model integrating temporal and spatial features. *Secur Commun Netw* 2021;2021:1–15.
- [43] Khan MA. HCRNNIDS: hybrid convolutional recurrent neural network-based network intrusion detection system. *Processes* 2021;9(5):834.
- [44] Goyal P, Goyal A. Comparative study of two most popular packet sniffing tools-tcpdump and wireshark. In: 2017 9th international conference on computational intelligence and communication networks (CICN). IEEE; 2017, p. 77–81.
- [45] Sommer R, Feldmann A. NetFlow: information loss or win? In: Proceedings of the 2nd ACM SIGCOMM workshop on internet measurement. IMW '02; New York, NY, USA: Association for Computing Machinery. 2002, 173–4.
- [46] Lashkari AH, Draper-Gil G, Mamun M SI, Ghorbani AA. Characterization of tor traffic using time based features. In: International conference on information systems security and privacy. 2017, p. 253–62.
- [47] Patator. Ftp patator and ssh patator <https://github.com/lanjelot/patator>.
- [48] Editcap.. <https://www.wireshark.org/docs/man-pages/editcap.html>.